HW 9 Write-Up

Riwaz Shrestha
Collaborators: Denali Schlesinger

My CSV_to_ndarray function takes in a path to the CSV file and returns two vectors of arrays. One is a 2d array of the one-hot encoded values, and the other is a 28x28 2d array of the numerical values representing each pixel. I use File and BufReader to open and read the files. I iterate through each line, take the first value, ohe, and store it in an array. For the rest of the line, I collect each value in an array and create an array using the vector. I then normalized the 28x28 2d array by normalizing it, with 1 being the highest value and all other values falling between 0 and 1.

I changed the initialization of the weights to get a wider range of values.

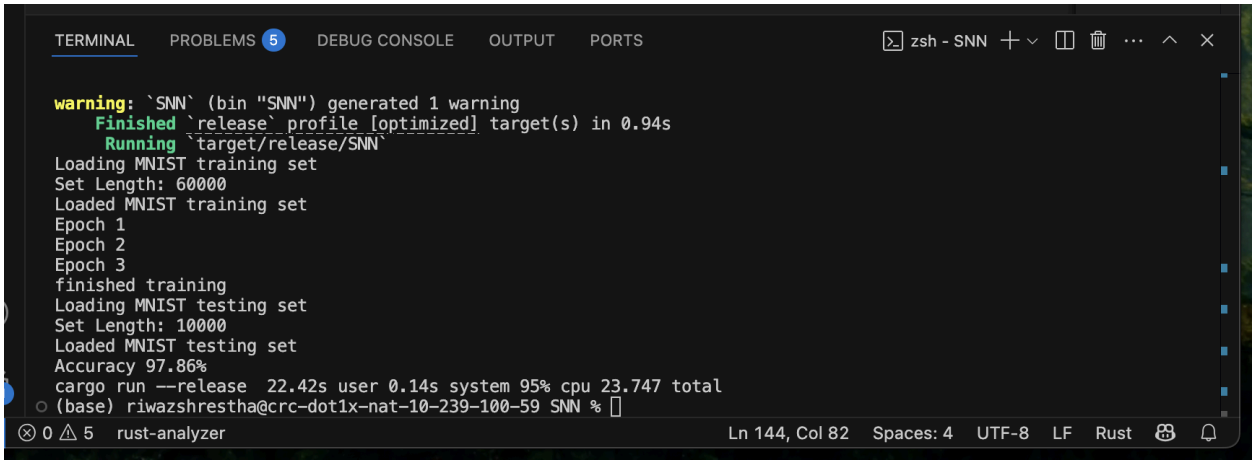
For the forward pass, I flattened the input array of normalized values, multiplied the first layer of weights by it, and then passed it through a ReLU closure. I repeated this process for the second and final layer, with the final layer going through softmax instead of ReLU. ReLU makes all negative values zero and leaves the other values unchanged. Softmax takes the resultant vector and makes it a probability vector. This function returns the layer 1, layer 2, and final output.

In the backward pass, I first calculate the error by subtracting the output vector from the ohe version of the value. I then find the gradient by taking the dot product of the error and the output of layer 2 transposed. Next, I calculate the derivative of the ReLU function and then calculate the error for the next layer of weights. I continue this process until I reach the initial input and the first matrix of weights. I then update the weight layers using the calculated error for each layer along with the learning rate.

For the main function, I first convert the CSV file, create the neural network, and then train it using each image 3 times (3 epochs). I then test my neural network by passing all the testing images through he neural network. For each output, I convert the output probability vector to its corresponding value and do the same for the ohe values to compare outputs. Lastly, I calculate the accuracy and print it.

I played around with the learning rate and initial weight range until I was satisfied with the accuracy it output. The combination I have consistently gets me over 97% accuracy. I also played around with the epoch and sizes of the hidden layers and got them to 99% accuracy.

I used 3 Blue 1 Brown's series on neural networks and ChatGPT to further my understanding of neural networks. My ChatGPT prompts were all just me asking what a derivative is and confirming the knowledge I got from 3B1B.


TERMINAL
PROBLEMS 5
DEBUG CONSOLE
OUTPUT
PORTS
zsh - SNN
warning: `SNN` (bin "SNN") generated 1 warning
Finished `release` profile [optimized] target(s) in 0.94s
Running `target/release/SNN`
Loading MNIST training set
Set Length: 60000
Loaded MNIST training set
Epoch 1
Epoch 2
Epoch 3
finished training
Loading MNIST testing set
Set Length: 10000
Loaded MNIST testing set
Accuracy 97.86%
cargo run --release 22.42s user 0.14s system 95% cpu 23.747 total
(base) riwazshrestha@crc-dot1x-nat-10-239-100-59 SNN %
0 5 rust-analyzer
Ln 144, Col 82
Spaces: 4
UTF-8
LF
Rust